PARALLEL & DISTRIBUTED COMPUTING · CLASS PROJECT

Biliran Resort & Tour Booking System

A parallel and distributed online and face-to-face booking system for resort and tour packages across the islands of Biliran, Philippines.

System Design and Initial Considerations

Document	Design v2 — flowcharts, conceptual framework, architecture, data model, requirements, risks, plan
Version / date	v2.0 · 29 October 2026 (incorporates adviser revisions)
Submission deadline	20 November 2026
Team	Member A — Central & integration · Member B — Island node · Member C — Mobile app · Member D — Consoles, reports & testing
Adviser	[Adviser name]
Repository	github.com/Abylon10/BookingSystem

Colour palette used across all user interfaces



#12212E · #307082 · #6CA3A2 · #ECE7DC · #EA9940 — Places are real; the resorts and tour operators named in this document are fictional.

Contents

1. Introduction
2. Objectives
3. Scope and limitations
4. Conceptual framework (IPO)
5. System architecture, modules, interfaces, data flow
6. Users, roles and data ownership
7. Use case diagram
8. Flowcharts
   8.1 Online booking
   8.2 Walk-in booking
   8.3 Arrival confirmation
   8.4 Mark service unavailable
   8.5 Catalog change by node admin
   8.6 Island registration and accounts
9. Sequence diagrams
10. Booking life cycle
11. Data model (ERD)
12. Concurrency, consistency and offline operation
13. Parallelism and experiments
14. Requirements
15. Technology and environment
16. Other initial considerations
17. Risks and mitigation
18. Project plan and team
19. Open questions for the adviser

1. Introduction

1.1 Background

Biliran is an island province in Eastern Visayas with tourism spread across the main island and smaller islands such as Maripipi and Higatangan. Resorts and tour operators on these islands sell to two kinds of customers at once: travellers who book ahead online, and walk-in guests who arrive at the front desk or the pier. Connectivity between the smaller islands and the mainland can be slow or interrupted, yet front desks must keep operating.

1.2 Problem

- **Double booking:** online and walk-in sales draw from the same rooms, seats and boat trips; without coordination the last unit can be sold twice.
- **Unreliable links:** a centralised system stops the island front desk whenever the internet link drops.
- **Fragmented management:** each resort maintains its own services and prices, while the province-wide view (registrations, accounts, reports) needs one place.
- **Slow aggregate queries:** searching or reporting across all islands one after another grows with every island added.

1.3 Proposed solution

A distributed system in which **each island runs its own node** that owns its services, packages, inventory and bookings, and a **central node** that serves the customer mobile app, registers islands and manages accounts. Nodes communicate by remote procedure calls (gRPC). Searches and reports run **in parallel** across nodes and worker threads, and every write is designed so that concurrent requests cannot oversell.

2. Objectives

2.1 General objective

To design and implement a parallel and distributed booking system for resort and tour packages in Biliran that supports online and face-to-face booking from a shared inventory without double booking, and continues to operate on an island when its connection to the central node is lost.

2.2 Specific objectives

1. Develop a mobile application (Expo / React Native) for customers to search packages, book, pay (simulated) and view their booking confirmation.
2. Develop an island node with a web console for receptionists (walk-in booking, arrival confirmation, marking services unavailable) and node admins (services, packages, prices, receptionist accounts).
3. Develop a central node with an admin panel for island registration, account management, and province-wide bookings and reports.
4. Guarantee consistency under concurrency: no inventory unit is sold twice, and each booking is checked in at most once.
5. Support partition tolerance: island operations, staff logins and local changes keep working offline and synchronise automatically on reconnection.
6. Apply and measure parallelism: parallel search fan-out across nodes and parallel report generation with worker threads, compared against sequential baselines.

3. Scope and limitations

3.1 Scope

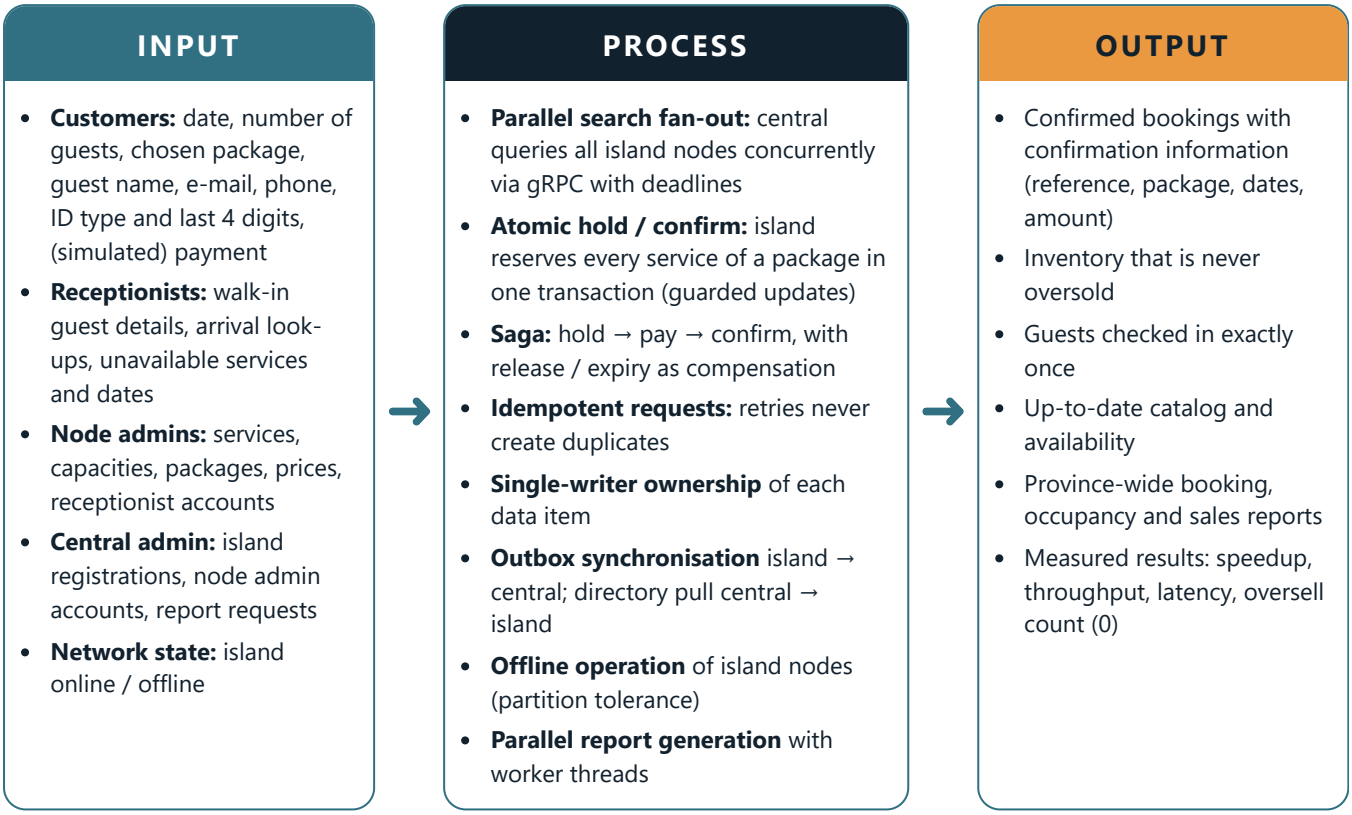
- Three island nodes (Biliran mainland, Maripipi, Higatangan) and one central node, demonstrated on four laptops connected by a phone hotspot.
- Package-based booking. Each package belongs to one island and is composed of services (room, tour, boat, meal).
- Online booking with a hold → pay → confirm flow; walk-in booking at the node console.
- Arrival confirmation by booking reference or name plus physical ID check (no tickets or QR codes).
- Manual blocking of unavailable services by the receptionist.
- Roles: central admin, node admin, receptionist, customer.
- Reports: bookings, occupancy and sales per island and package.

3.2 Limitations

- **Payment is simulated.** No real payment gateway (e.g. GCash, cards) is integrated.
- Packages cannot combine services from different islands.
- While an island is offline, its packages are **not sold online** (see §12.3).
- Customers have no accounts; bookings are looked up by reference and e-mail.
- Only the last four digits of a guest's ID are recorded; identity is verified visually by staff.
- Demonstrated on a local hotspot network; production hosting, TLS between nodes and backups are out of scope.
- Resorts and tour operators are fictional; prices and capacities are sample data.

4. Conceptual framework

The study follows the Input–Process–Output (IPO) model. Inputs from the four user groups are processed by the distributed nodes using parallel and concurrency-safe techniques, producing consistent bookings, confirmations and reports. Evaluation results feed back into the design.



4.1 Key concepts applied

Concept	How it appears in the system
Distribution / data partitioning	Data partitioned by island; each node stores and owns its own island's data.
Remote procedure call	gRPC contract (`proto/booking.proto`) between central and island nodes.
Parallelism (task)	Concurrent RPCs to all islands during search; total time ≈ slowest island instead of the sum.
Parallelism (data)	Report data split across worker threads (1 / 2 / 4 / 8) and merged.
Concurrency control	Atomic conditional updates and transactions; single writer per data item.
Fault tolerance / partition	Deadlines, offline mode, outbox with retries, de-duplication by event id.
Consistency trade-off (CAP)	Consistency chosen over availability for online sales of an unreachable island.

5. System architecture

Four laptops and the customers' phones share one phone hotspot. Laptop 1 is the central node; Laptops 2–4 each run one island node. The same island software runs on every island laptop with a different island id.

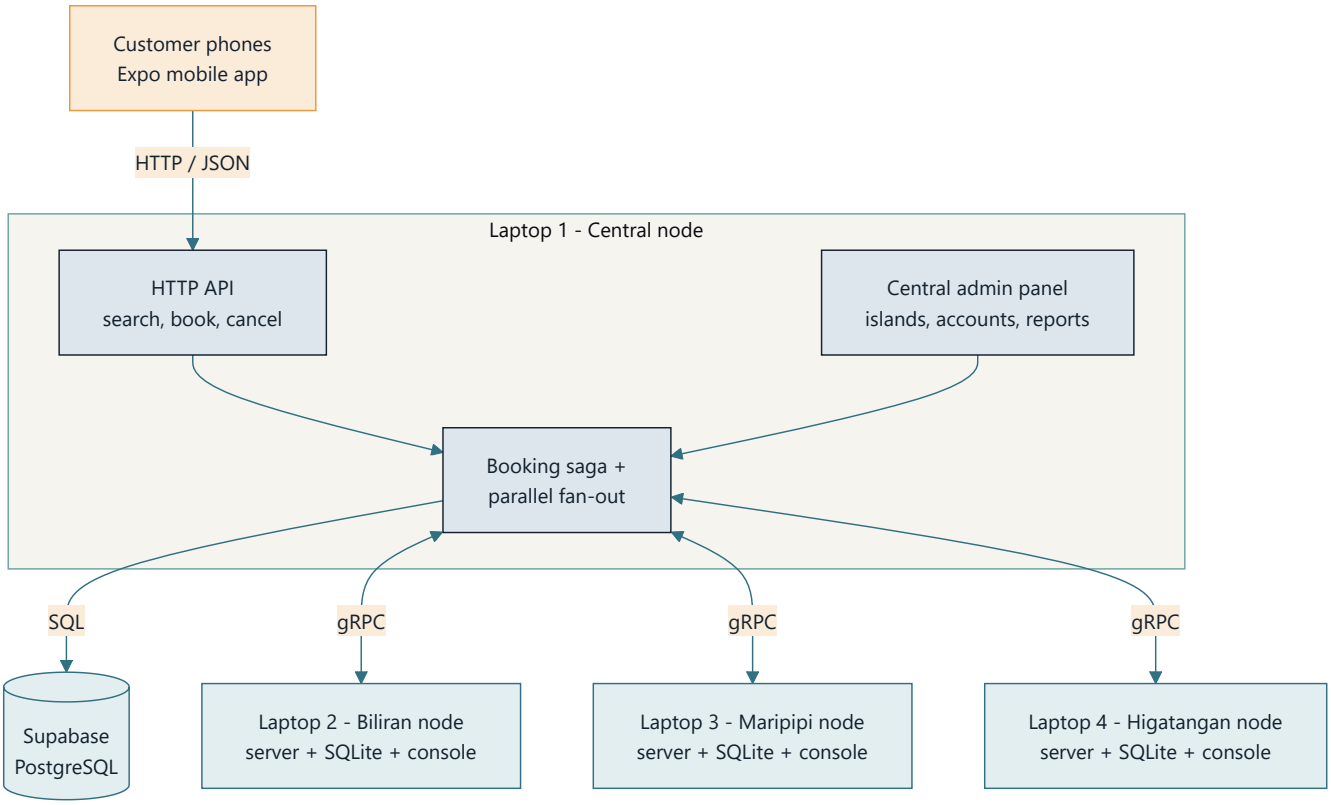


Figure 1. Distributed architecture and communication paths

Component	Runs on	Responsibilities
Mobile app	Customer phones (Expo Go)	Search packages, book, simulated payment, confirmation info, my bookings, cancel
Central node	Laptop 1	HTTP API for the app, parallel fan-out, booking saga, central admin panel, island registration, admin accounts, copies of all bookings for reports
Supabase (PostgreSQL)	Cloud, used by central only	Central storage: islands, accounts, catalog copy, booking copy, sync events
Island node	Laptops 2–4	Services, packages, inventory, bookings (single owner), receptionist accounts, outbox, node console
Node console	Browser on each island laptop	Login; receptionist pages; node admin pages

5.1 Modules

ID	Module	Runs on	Responsibility	Owner
M1	Customer mobile app	Phones	Search packages, book, simulated payment, confirmation info, my bookings, cancel	Member C
M2	Central API	Central	HTTP endpoints for the app; input validation; idempotency keys	Member A
M3	Booking orchestrator	Central	Parallel fan-out search; hold → pay → confirm saga; release on failure	Member A
M4	Central admin panel	Central	Island registration, node admin accounts, all bookings, reports	Member A
M5	Central sync service	Central	Receives island events (`PushEvents`); serves directory changes (`PullDirectory`)	Member A
M6	Island booking engine	Each island	Inventory, packages, hold/confirm/release, walk-in booking, arrival confirmation, blocking	Member B
M7	Island catalog & accounts	Each island	Services, packages, prices; receptionist accounts; local login (scrypt)	Member B
M8	Island sync agent	Each island	Outbox sender with retries; directory puller; partition switch	Member B
M9	Node console	Island laptops	Receptionist pages and node admin pages (web)	Member D
M10	Report workers	Central	Occupancy and sales reports computed in parallel with worker threads	Member D
M11	Test and benchmark tools	Any laptop	Race tests, load tests, sequential vs parallel benchmarks	Member D

5.2 Interfaces

Interface	Protocol	Operations
App → Central (M1→M2)	HTTP / JSON	`GET /api/packages?day&guests` · `POST /api/bookings` (hold) · `POST /api/bookings/:id/pay` · `GET /api/bookings/:ref?email` · `POST /api/bookings/:id/cancel`
Central → Island (M3→M6)	gRPC `IslandService`	`GetCatalog` · `SearchAvailability` · `HoldBooking` · `ConfirmBooking` · `ReleaseHold` · `CancelBooking` · `GetBooking` · `Ping`
Island → Central (M8→M5)	gRPC `CentralService`	`PushEvents` (bookings, catalog changes, receptionist accounts) · `PullDirectory` (registration, admin accounts)
Console → Island (M9→M6/M7)	HTTP / JSON	`/api/login` · `/api/walkin` · `/api/arrivals` (search, confirm) · `/api/blocks` · `/api/services` · `/api/packages` · `/api/staff` · `/api/partition`
Admin panel → Central (M4)	HTTP / JSON	`/admin/api/islands` · `/admin/api/accounts` · `/admin/api/bookings` · `/admin/api/reports`
Central → Supabase	PostgreSQL (supabase-js)	Islands, accounts, catalog copy, booking copy, events

5.3 Data flow

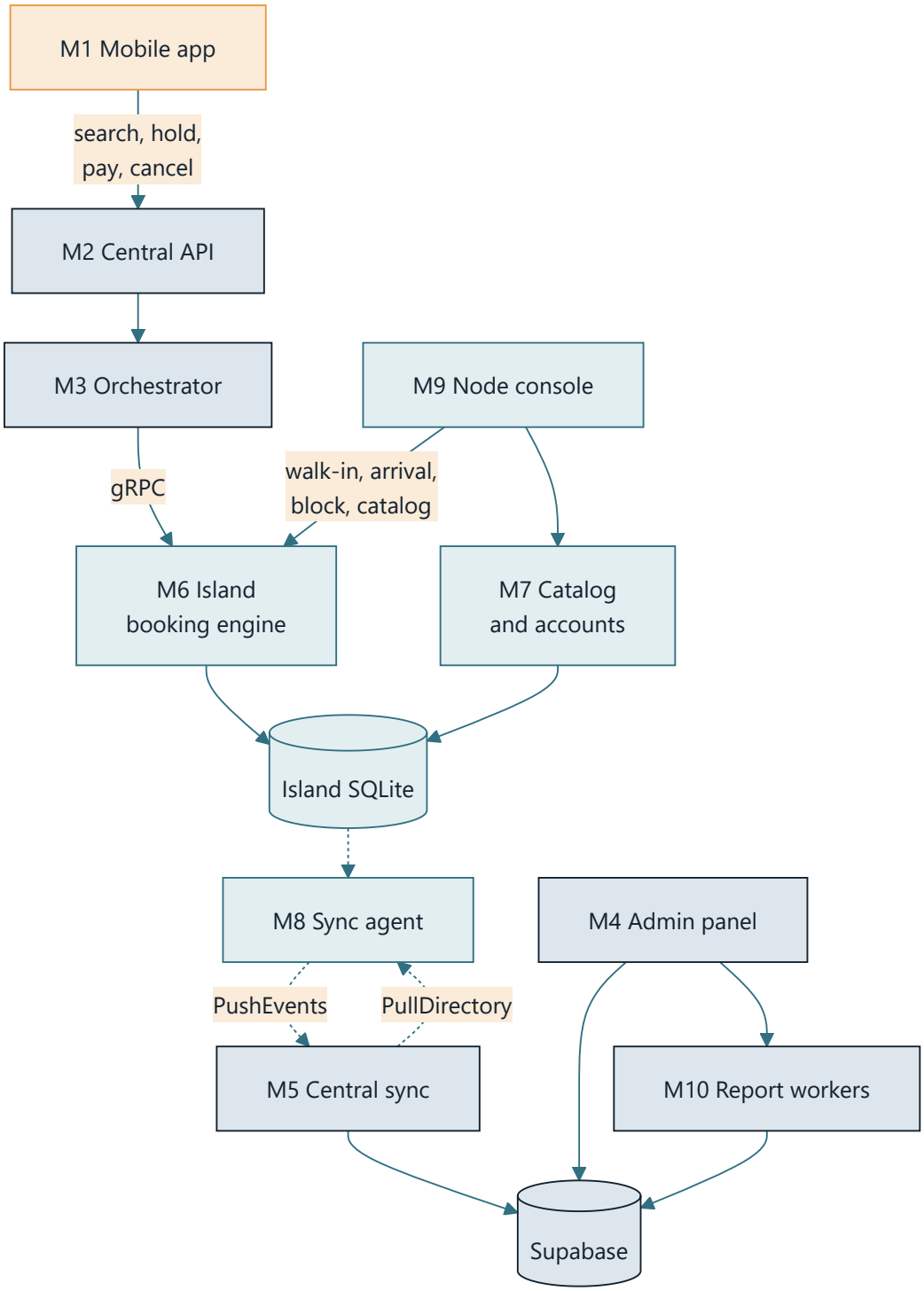


Figure 1b. Main data flows. Solid arrows: requests in the booking path; dashed: background synchronisation.

1. **Booking path:** app → central API → orchestrator → owning island's booking engine → island database. The answer returns the same way.
2. **Walk-in and arrival path:** node console → island booking engine → island database (no central involvement).
3. **Island → central:** every local change also writes an outbox event; the sync agent pushes it to central, which stores copies in Supabase.
4. **Central → island:** registration and admin accounts flow down when the island pulls the directory.
5. **Reports:** admin panel → report workers read the booking copies in parallel.

6. Users, roles and data ownership

6.1 Roles

Role	Works on	Permissions
Central admin	Central admin panel	Register and edit islands/resorts; create and disable node admin accounts; view all bookings; generate reports
Node admin	Own island's node console	Manage own island's services, capacities, packages and prices; create and disable receptionist accounts; all receptionist functions
Receptionist	Own island's node console	Walk-in bookings; arrival confirmation; mark services unavailable for given dates; view the island's bookings
Customer	Mobile app	Browse and book packages; simulated payment; view confirmation; look up and cancel own booking (reference + e-mail)

6.2 Data ownership (single writer)

Every data item has exactly one owner, the only node allowed to change it. Other nodes keep read-only copies. This removes conflicting edits by design.

Data	Owner	Copy kept by	Copy updated by
Island registration	Central	Island	Island pulls (`PullDirectory`)
Central and node admin accounts	Central	Island	Island pulls (`PullDirectory`)
Receptionist accounts	Island	Central	Island outbox → `PushEvents`
Services, packages, prices	Island	Central	Island outbox → `PushEvents`
Inventory and blocked units	Island	—	Never copied; always queried live
Bookings (online and walk-in)	Island	Central	Island outbox → `PushEvents`

6.3 Sample catalog (fictional businesses, real places)

Island	Services	Example packages
Biliran mainland	Lagundi Bay Resort rooms; Tinago & Ulan-Ulan Falls trek; van transfer	Falls Day Trek; 2D1N Lagundi Bay + Falls
Maripipi	Dagami Reef cottages; Sambawan island hopping; boat transfer	Sambawan Day Tour; Sambawan Overnight (cottage + hopping + boat)
Higatangan	Sandbar Inn rooms; rock formations & sandbar tour; pump boat from Naval	Sandbar Day Trip; 2D1N Higatangan Escape

7. Use case diagram

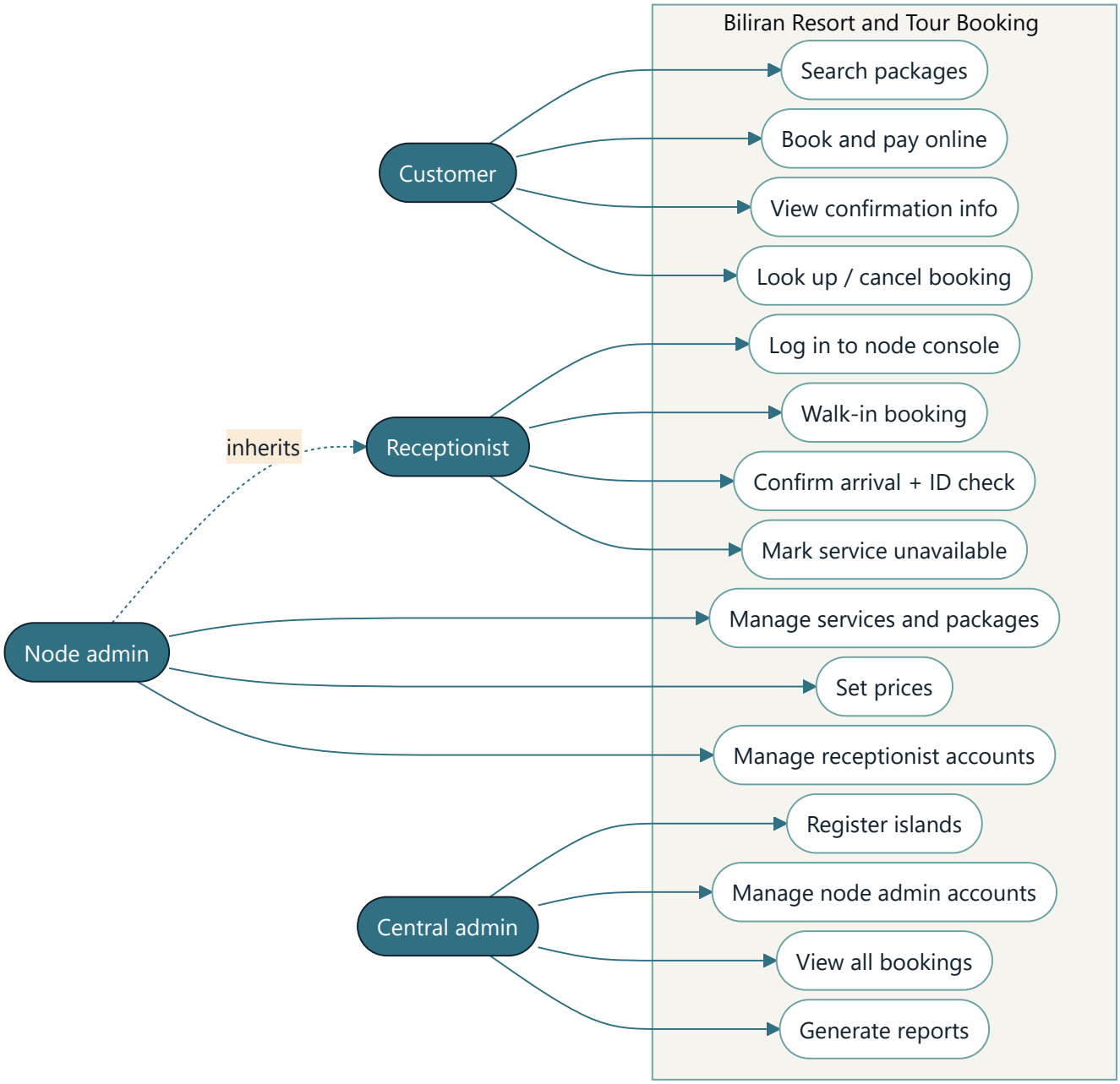


Figure 2. Actors and use cases (node admin inherits all receptionist use cases)

8. Flowcharts

8.1 Online booking (customer → central → island)

Part 1 — Parallel search

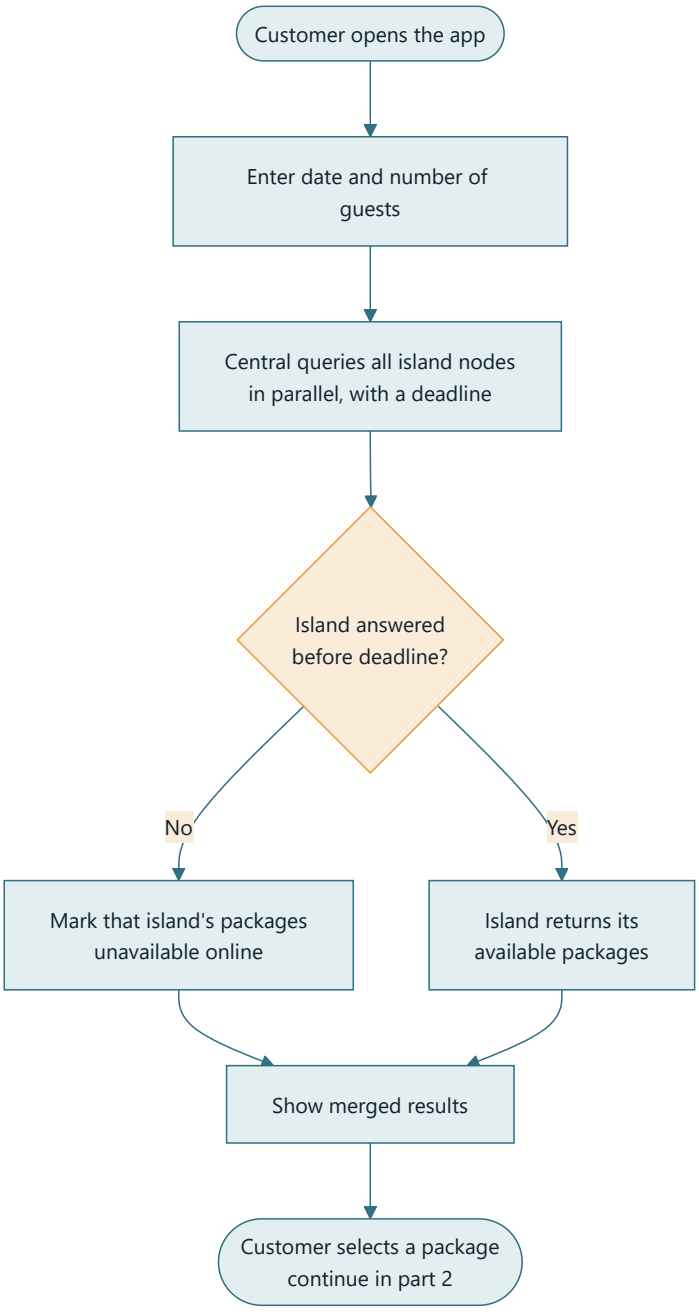


Figure 3a. Central asks every island at the same time; an island that misses the deadline is shown as unavailable

Part 2 — Hold, pay, confirm (saga)

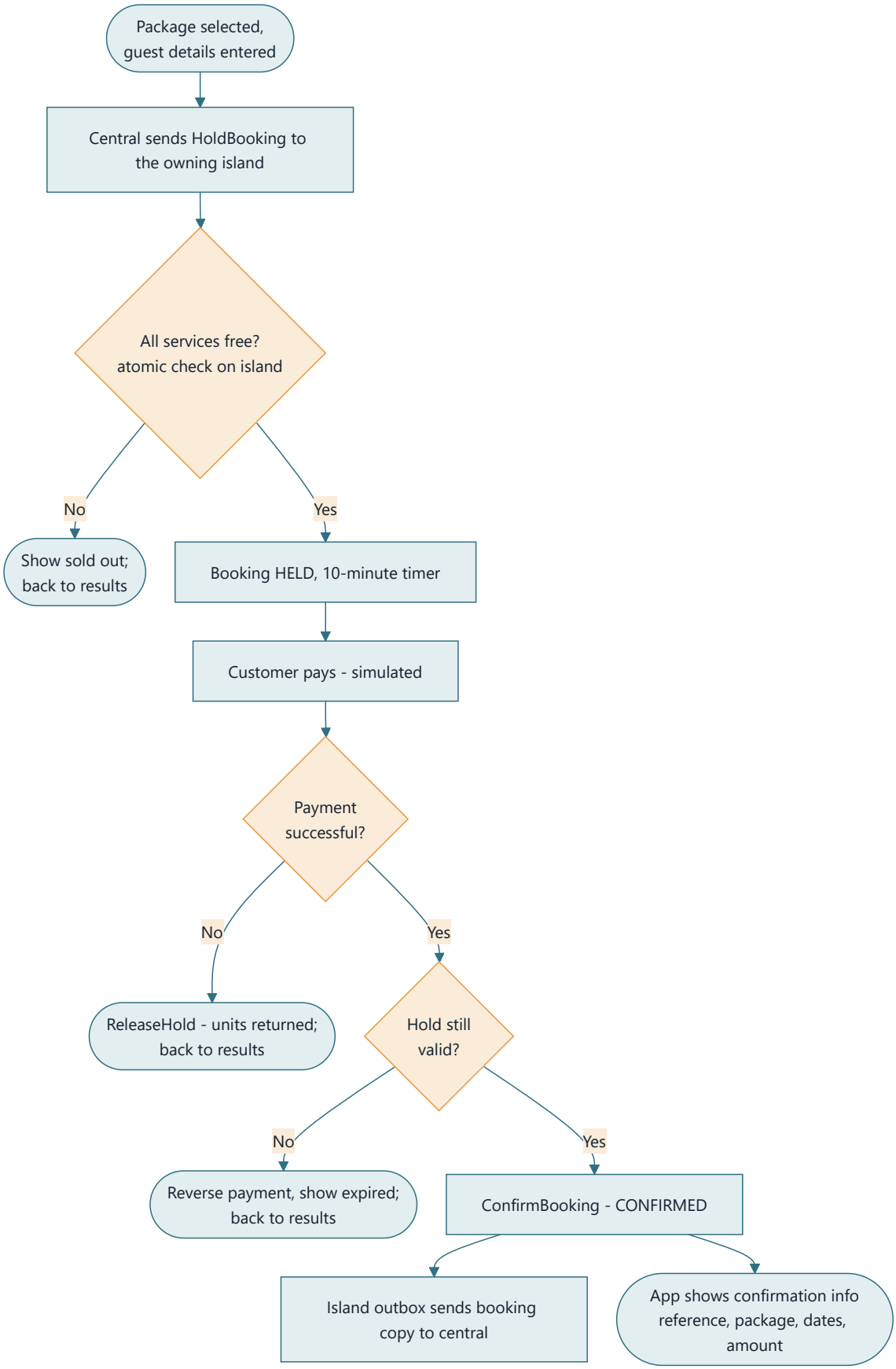


Figure 3b. The owning island reserves all services atomically; release or expiry undoes the hold

8.2 Walk-in booking (receptionist)

> Online and walk-in bookings both go through the island node that owns the inventory, so the race for the last unit is decided by one atomic update on one database.

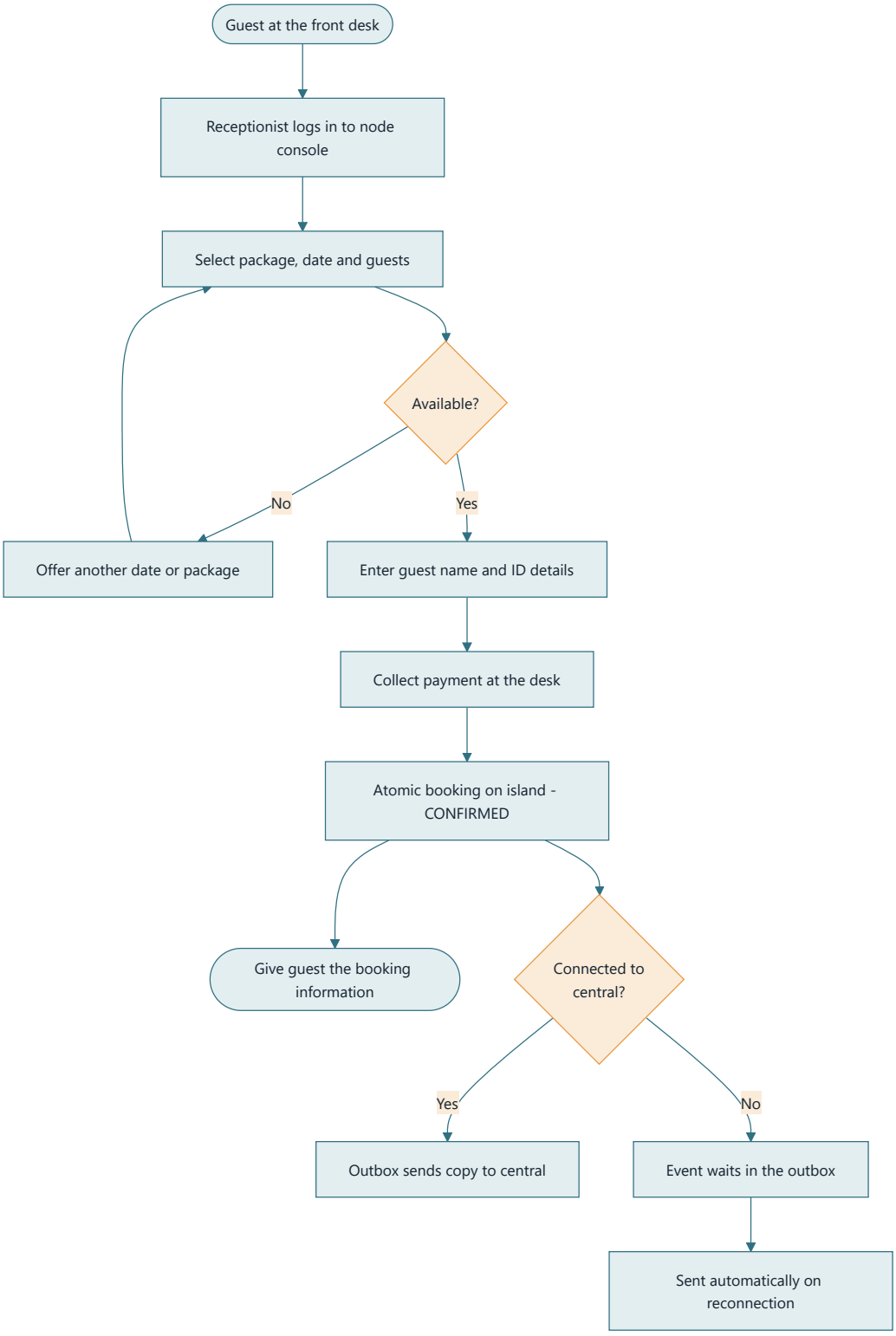


Figure 4. Walk-in booking from the same inventory as online booking

8.3 Arrival confirmation (no tickets)

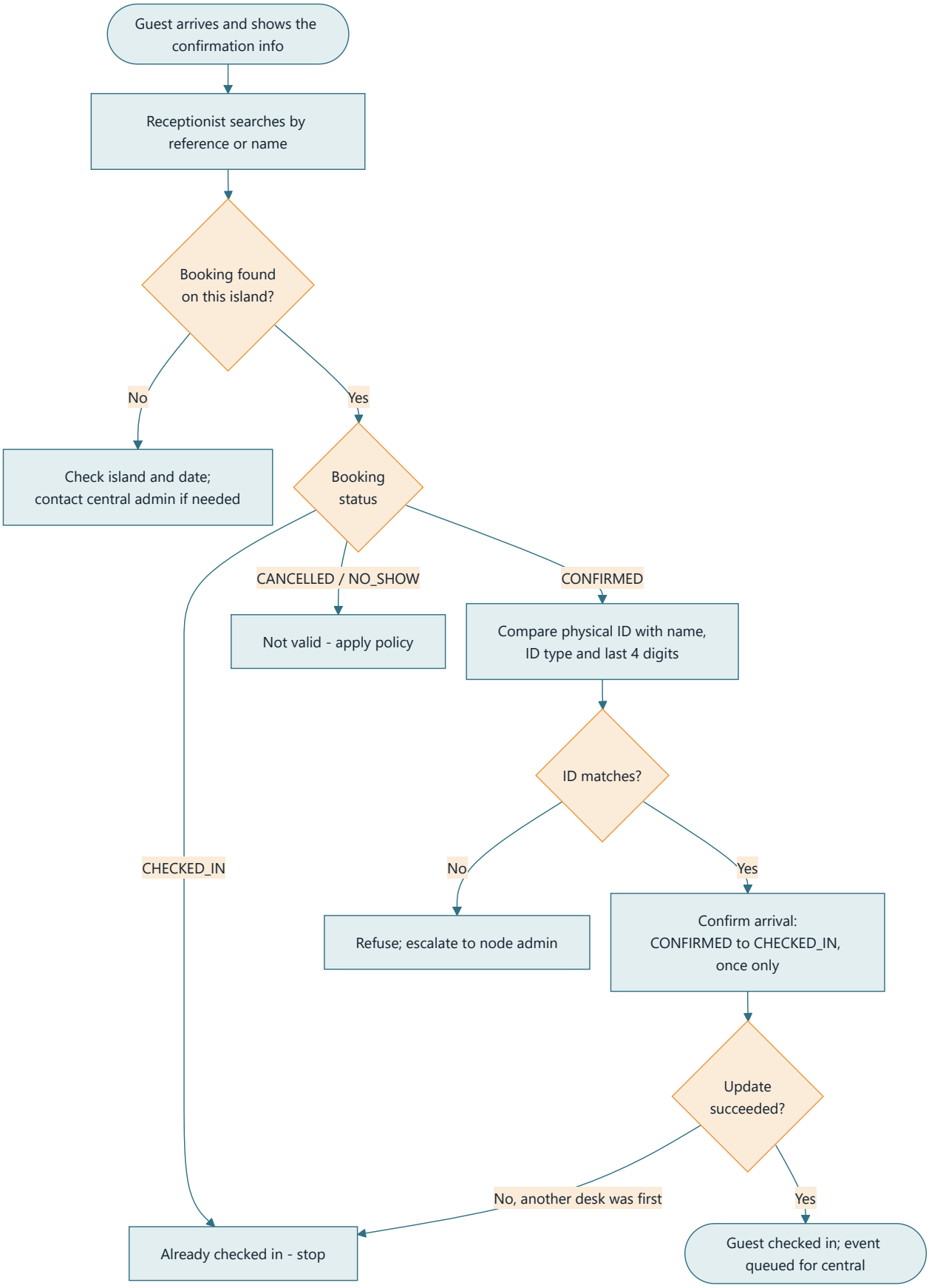


Figure 5. The guest shows the confirmation information received after payment; staff verify the ID

8.4 Mark service unavailable (receptionist)

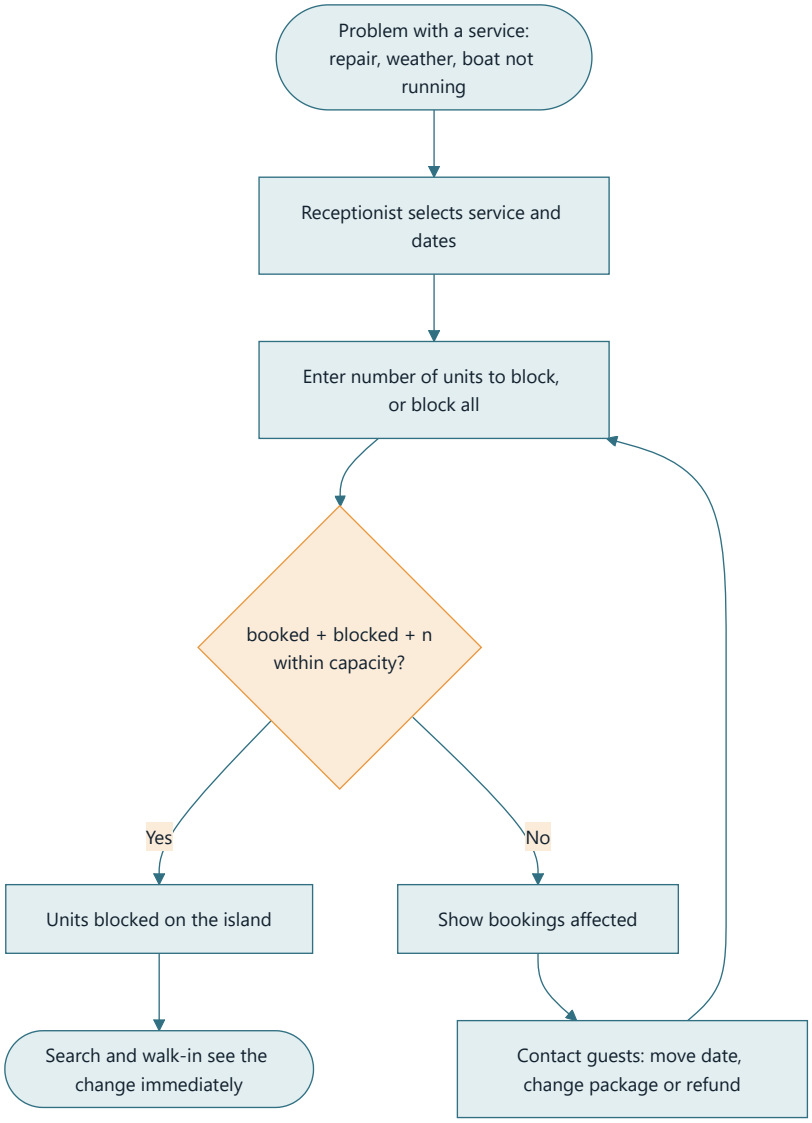


Figure 6. Manual input of unavailable services (repairs, weather, boat not running)

8.5 Catalog change by node admin

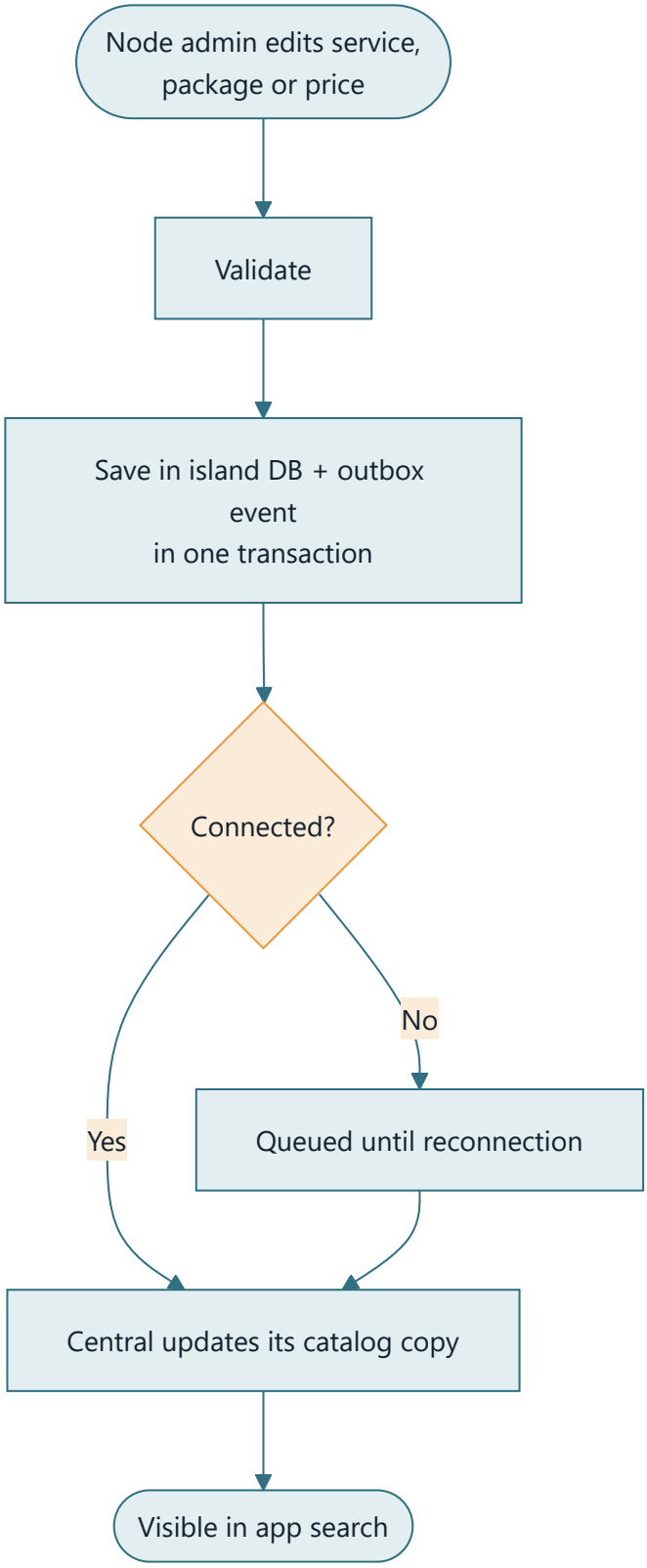


Figure 7. Changes are saved locally first (works offline), then synchronised to central

8.6 Island registration and accounts

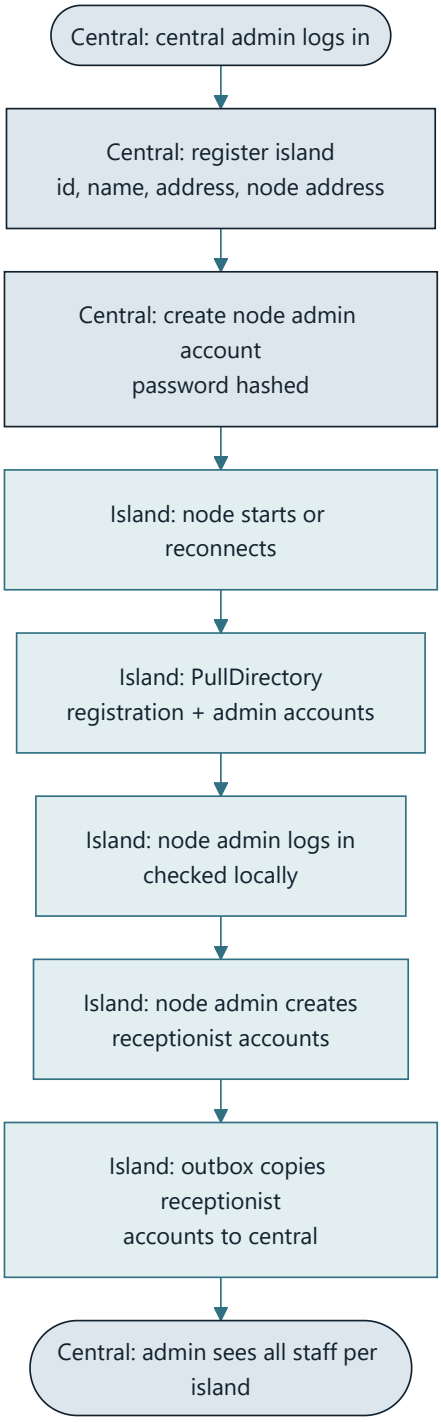


Figure 8. Central admin registers islands and node admins; node admins create receptionists (dark = central, teal = island)

Login

Passwords are stored only as salted scrypt hashes. Each node verifies logins against its local copy of the accounts, so staff can log in while the island is offline. Sessions expire after inactivity; disabled accounts stop working at the next directory pull.

9. Sequence diagrams

9.1 Online booking across nodes

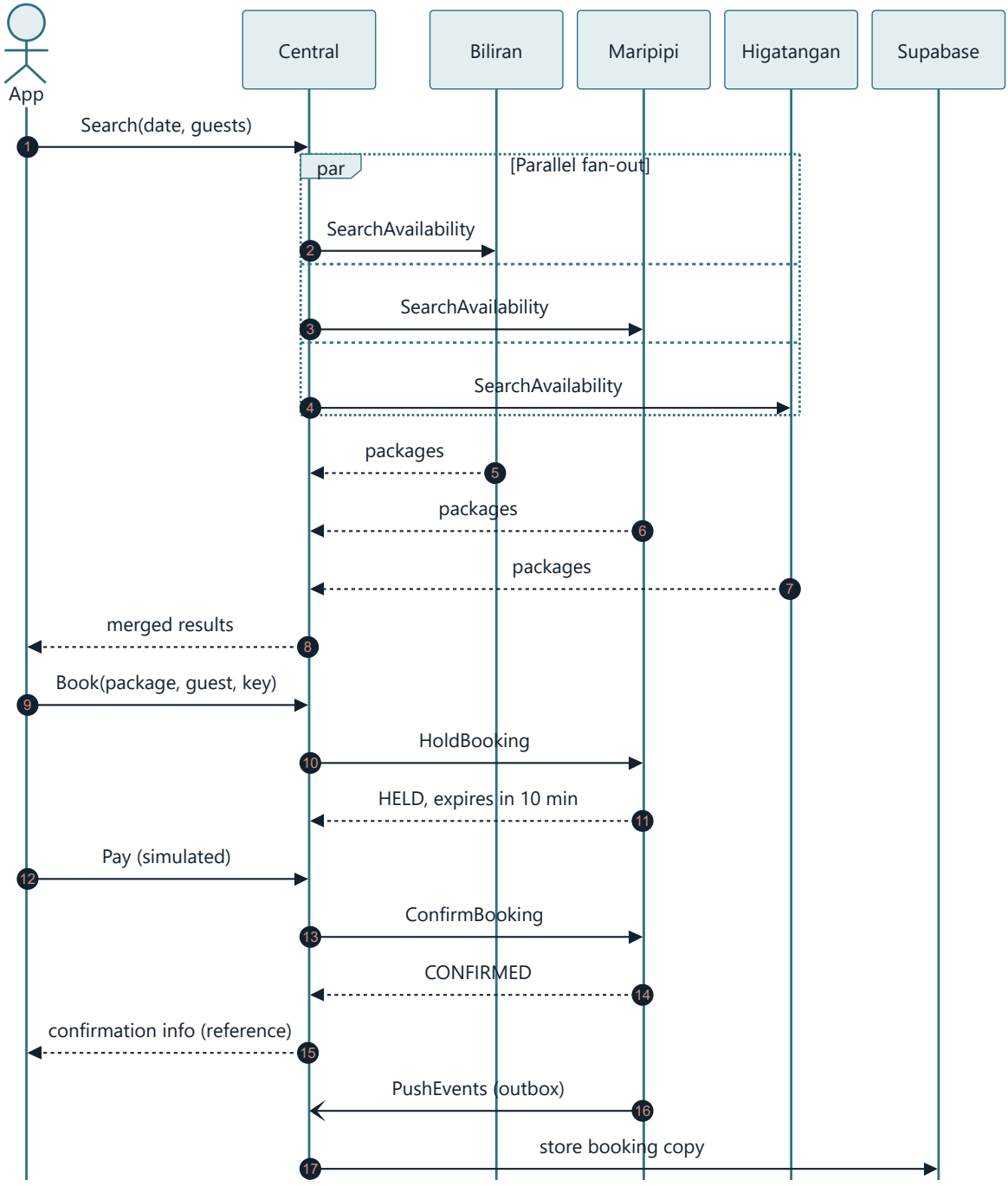


Figure 9. Parallel fan-out (par block), then hold, payment and confirmation on the owning island

9.2 Island offline and recovery

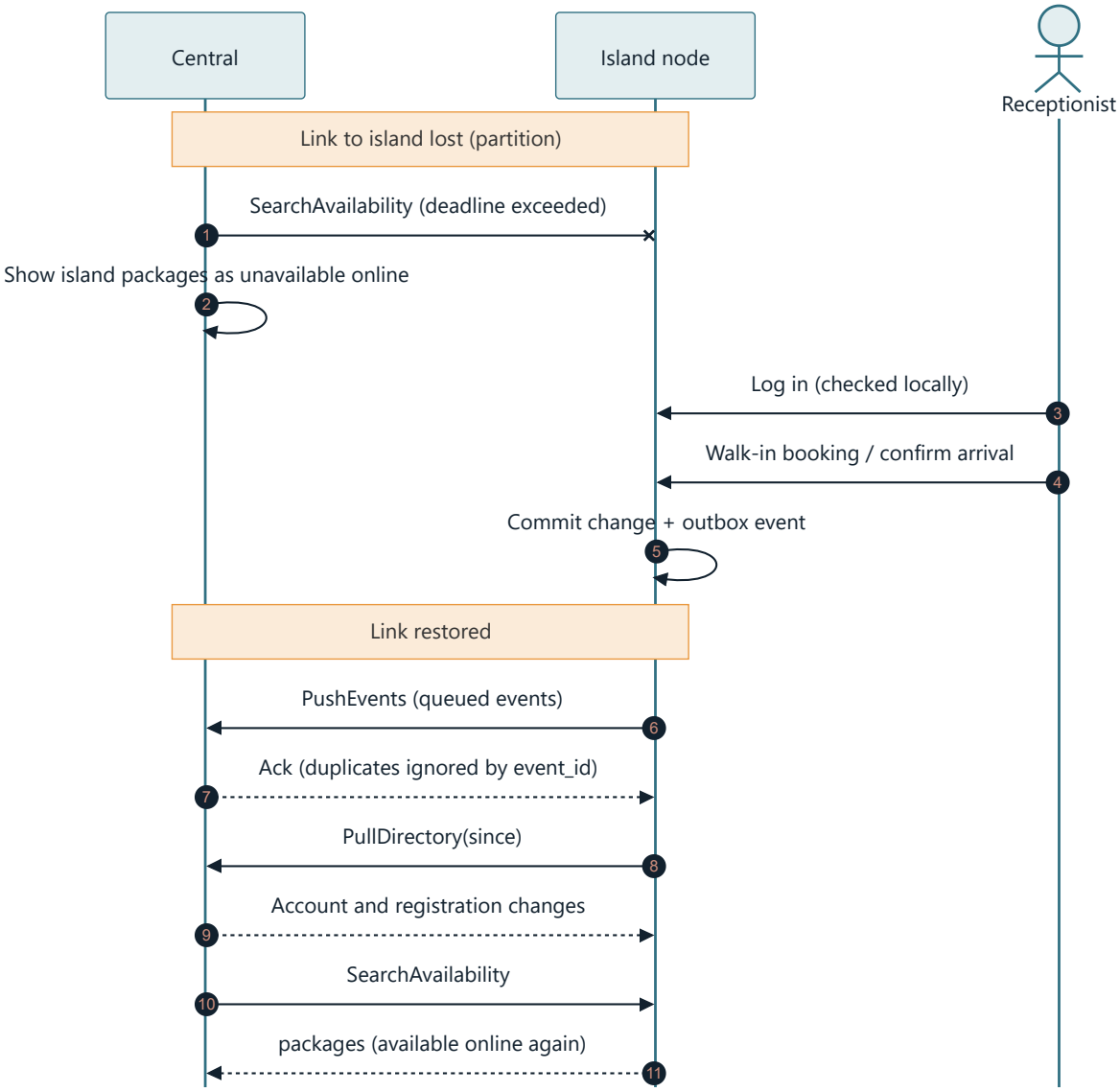


Figure 10. Partition: the island keeps working; queued events synchronise on reconnection

10. Booking life cycle

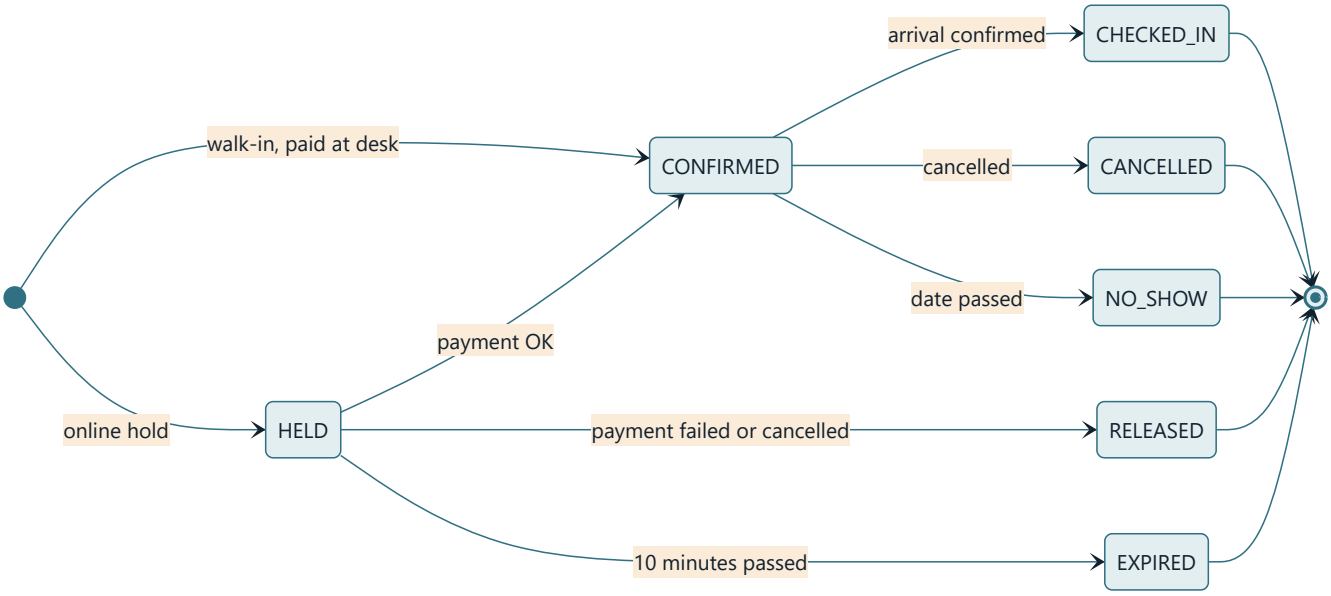


Figure 11. Booking states. Each transition is a single conditional update, so it can happen only once.

State	Meaning
HELD	Online booking: inventory reserved, waiting for payment (expires after 10 minutes)
CONFIRMED	Paid (online) or paid at the desk (walk-in); guest expected
CHECKED_IN	Guest arrived and identity verified by staff
RELEASED / EXPIRED	Hold undone because payment failed, was cancelled, or took too long; units returned
CANCELLED	Cancelled by the customer or staff; units returned; refund per policy
NO_SHOW	Date passed without check-in (set by a nightly batch job)

11. Data model

11.1 Island node database (SQLite, one per island)

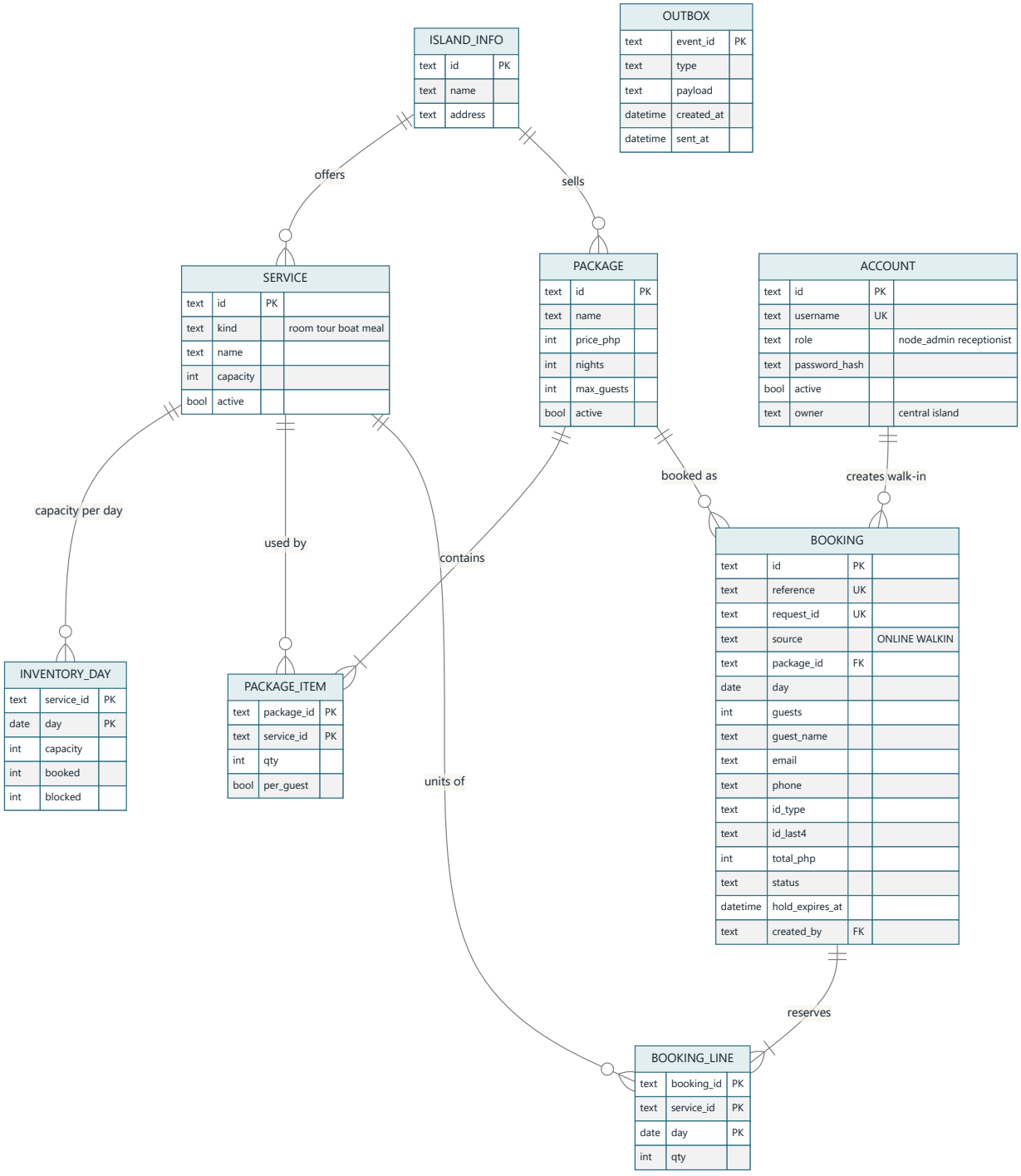


Figure 12. Island entities. `booking_line` records exactly which units a booking took, so cancellation and release return the same units.

> **Inventory rule.** For every service and day: **booked + blocked ≤ capacity**. Booking a package increases `booked` on every service and day it uses, inside one transaction, each step guarded by `WHERE booked + blocked + qty <= capacity`. If any step fails, the whole transaction is rolled back. Marking units unavailable increases `blocked` under the same guard.

11.2 Central database (Supabase PostgreSQL)

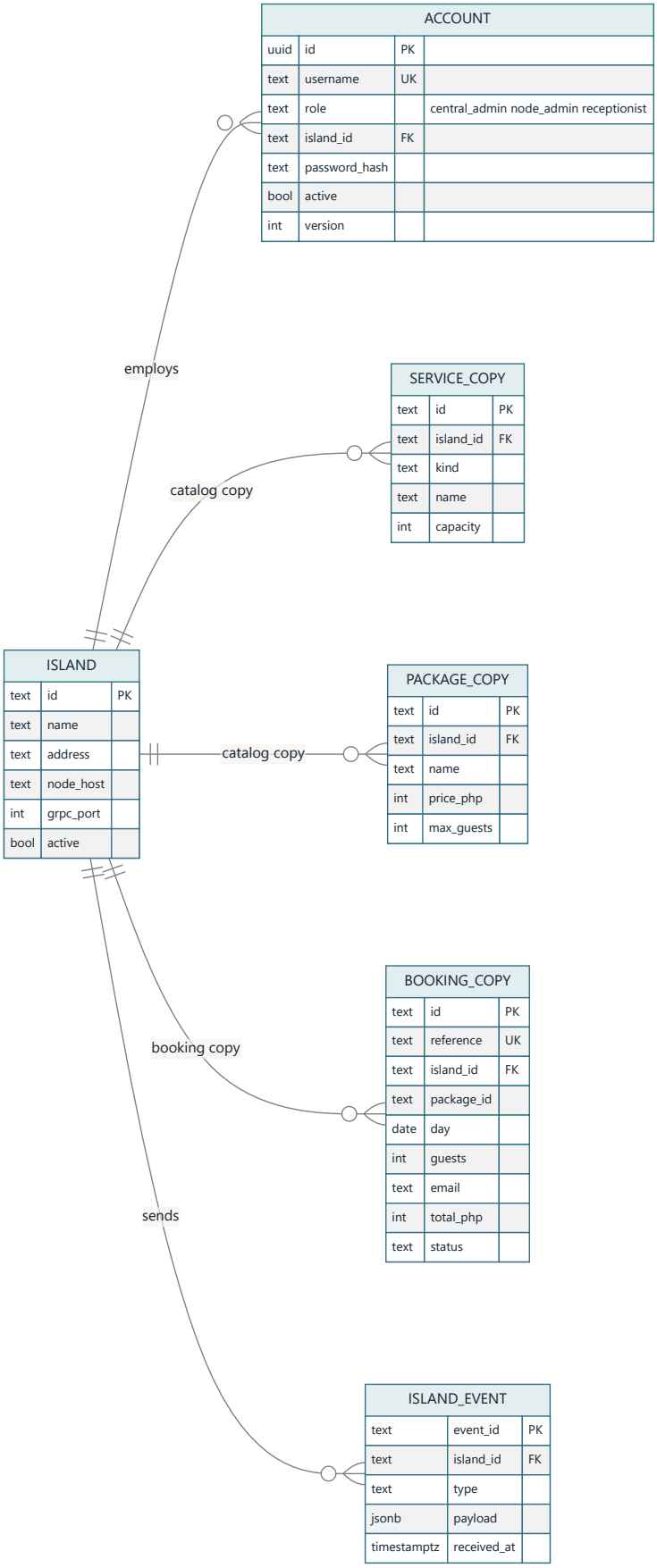


Figure 13. Central entities. Catalog and booking tables are read-only copies fed by island events.

12. Concurrency, consistency and offline operation

12.1 Race conditions and their solutions

Race	Where	Solution
Online buyer vs walk-in guest for the last room	Island	Both requests go to the owning island; one guarded atomic update decides; the loser is told "sold out" before paying
Many online buyers for the last seats	Island	Same guarded update; transactions serialise writers (`BEGIN IMMEDIATE`)
Receptionist blocks a room while it is being booked	Island	Both use the same rule (booked + blocked ≤ capacity); the second to commit fails
Hold expires while the customer is paying	Island	Confirm succeeds only on a non-expired HELD booking; otherwise the payment is reversed
App or central retries after a timeout	Central, island	Idempotency key (`request_id` , unique) returns the same booking
Two desks confirm the same arrival	Island	`UPDATE … WHERE status = 'CONFIRMED'` succeeds once
Event delivered twice during sync	Central	Events de-duplicated by `event_id`
Central admin and node admin edit the same data	—	Impossible: single owner per data item (§6.2)

12.2 Behaviour while an island is offline

Situation	Behaviour
Island's packages in the mobile app	Shown as **unavailable**; central cannot hold or confirm, so it does not sell
Walk-ins, arrivals, marking unavailable	Continue normally on the island
Node admin changes, new receptionists	Saved locally and queued in the outbox
Staff login	Works (accounts stored on the node)
Online bookings already confirmed	Can be checked in (stored on the island)
On reconnection	Outbox drains to central; island pulls account and registration changes

12.3 Consistency trade-off

Under a network partition the system cannot keep both online availability and consistency for an island's inventory (CAP theorem). Because there is no separate online allotment, only the island can decide whether a unit is free. The design therefore **chooses consistency**: it never sells online what the owning island cannot confirm, while the island itself stays fully available to walk-in guests.

The partition is demonstrated with a **"Disconnect from Central"** switch on each node console (and optionally a firewall rule blocking the central laptop).

13. Parallelism and experiments

13.1 Where parallelism is used

Technique	Description
Parallel RPC fan-out	Central sends search requests to all island nodes at once and waits for all, with a deadline per call so one slow island cannot block the result.
Concurrent request handling	Every node serves many clients at the same time; correctness under concurrency comes from atomic guarded updates.
Data-parallel reports	Booking records are split into chunks processed by worker threads (occupancy, sales per island and package), then merged.
Parallel load generation	Test clients send N simultaneous requests (online and walk-in mixed) to measure throughput and verify zero oversell.

13.2 Experiments

Experiment	Compare	Metrics
Search fan-out (3 islands)	Sequential vs parallel RPC	Latency, speedup (ideal ≈ 3×; prototype measured 2.93× with 150 ms simulated link)
Last-unit race	N = 10 / 30 / 100 clients, online + walk-in	Bookings vs units available (oversell must be 0), throughput
Double arrival confirmation	Two desks at the same instant	Exactly one success
Report generation	1 / 2 / 4 / 8 worker threads	Speedup S(p) = T(1) / T(p), efficiency E(p) = S(p) / p
Load test	10 / 50 / 100 concurrent clients	Throughput, p95 latency, error rate
Partition recovery	Island offline for 1 / 5 minutes	Events queued, time to fully synchronise after reconnection

13.3 Expected limit (Amdahl's law)

Speedup with *p* workers is bounded by **S(p) = 1 / ((1 − f) + f / p)**, where *f* is the parallelisable fraction. Reading the data and merging results remain sequential, so report speedup is expected to flatten as threads increase. The measured curve will be compared with this bound.

14. Requirements

14.1 Functional requirements

ID	Actor	Requirement
FR-01	Customer	Search packages by date and number of guests across all islands, with each island's online status shown.
FR-02	Customer	View package details: services included, nights, price, maximum guests, island.
FR-03	Customer	Book a package with guest name, e-mail, phone, ID type and last 4 digits; pay (simulated).
FR-04	Customer	Receive confirmation information (reference, package, dates, guests, amount) after payment.
FR-05	Customer	Look up and cancel a booking using reference and e-mail.
FR-06	Receptionist	Log in to the node console (also while offline).
FR-07	Receptionist	Create walk-in bookings from the shared inventory.
FR-08	Receptionist	Search bookings by reference or name and confirm arrival after the ID check.
FR-09	Receptionist	Mark service units unavailable for selected dates and see affected bookings.
FR-10	Node admin	Create, edit and disable services (with capacity) and packages (with price and items).
FR-11	Node admin	Create and disable receptionist accounts for the island.
FR-12	Central admin	Register and edit islands; create and disable node admin accounts.
FR-13	Central admin	View all bookings filtered by island, status and date.
FR-14	Central admin	Generate occupancy and sales reports per island and package.
FR-15	System	Expire unpaid holds after 10 minutes; mark no-shows after the booking date.
FR-16	System	Synchronise island changes to central and directory changes to islands automatically.

14.2 Non-functional requirements

Quality	Requirement
Consistency	Zero oversold units and zero double check-ins under concurrent load.
Availability	Island operations continue during loss of connection to central.
Performance	Search across 3 islands in under 1 s on the hotspot; parallel faster than sequential.
Reliability	No synchronisation event lost; duplicates have no effect.
Security	Hashed passwords; role-based access; central database key kept only on the central laptop.
Privacy	Minimal personal data; only the last 4 digits of IDs stored (Data Privacy Act of 2012, RA 10173).
Usability	Mobile-first app; consoles usable on a laptop screen; consistent palette.
Maintainability	One code base per role; every code block commented; shared gRPC contract.

15. Technology and environment

15.1 Software stack

Layer	Technology	Reason
Mobile app	Expo SDK 58 (React Native, TypeScript)	One code base for Android and iPhone; runs in Expo Go without app-store builds
Central and island servers	Node.js 24+ with TypeScript, Express	Same language as the app; runs TypeScript directly
Node-to-node communication	gRPC (`@grpc/grpc-js` , Protocol Buffers)	Typed RPC contract, deadlines, efficient binary protocol
Island storage	SQLite (built-in `node:sqlite`)	Embedded, works offline, transactional
Central storage	Supabase (PostgreSQL)	Hosted database with dashboard; row-level security
Consoles / admin panel	HTML, CSS, JavaScript served by each node	No installation on staff laptops
Parallel reports	`node:worker_threads`	True parallel CPU work in Node.js
Passwords	`node:crypto` scrypt	Memory-hard password hashing, no extra dependency
Version control	Git + GitHub	Team collaboration, history

15.2 Hardware and network

Item	Specification
Laptops (4)	Windows, Node.js 24+, modern browser; one central, three island nodes
Phones (4)	Android or iPhone with Expo Go supporting SDK 58
Network	One phone hotspot (all 8 devices); mobile data needed only for Supabase
Ports	Central HTTP 4000 / gRPC 50050; islands HTTP 8081–8083 / gRPC 50051–50053

15.3 Repository layout

Folder	Contents
`app/`	Expo mobile app
`central/`	Central server, admin panel, Supabase migrations
`island/`	Island node server, SQLite, node console
`proto/`	gRPC contract
`shared/`	Configuration and RPC helpers used by all servers
`loadtest/`	Race, search and load experiments
`config/`	Network addresses and seed data

16. Other initial considerations

16.1 Network and demo setup

- Phone hotspots often allow only 5–10 devices; test with all 8 connected early.
- Some hotspots and school Wi-Fi block device-to-device traffic (client isolation); verify a phone can open the central laptop's address.
- Allow Node.js through Windows Firewall on private networks on every laptop.
- Record each laptop's IP address in `config/network.hotspot.json`; IPs can change when reconnecting.
- Install all dependencies before demo day; only Supabase needs mobile data during the demo.

16.2 Security and privacy

- The Supabase secret key stays in `.env` on the central laptop only; it is never committed or put in the app.
- Passwords hashed with scrypt; accounts can be disabled; staff see only their own island.
- Collect only what is needed: name, e-mail, phone, ID type and last 4 digits (RA 10173 — data minimisation).
- Traffic on the hotspot is unencrypted in the prototype; TLS between nodes is listed as future work.

16.3 Business rules to agree on

- Hold time before payment (proposed 10 minutes).
- Cancellation and refund policy (e.g. full refund up to 48 hours before; none after).
- Package pricing: per package up to a maximum number of guests (proposed) or per person.
- Check-in window and when a booking becomes a no-show.
- What happens to bookings affected when a service is marked unavailable (move date, change package, refund).

16.4 Testing plan

- **Unit level:** booking transaction, blocking rule, state transitions, password hashing.
- **Integration:** app → central → island booking, cancellation, sync, directory pull.
- **Concurrency:** race tests for last unit and double check-in; mixed online and walk-in.
- **Partition:** disconnect an island, operate, reconnect, verify synchronised state.
- **Performance:** sequential vs parallel search, worker-thread report speedup, load test.
- **User acceptance:** receptionist and admin walk-through with the consoles.

16.5 Demo script (outline)

1. Central admin registers an island and its node admin; node admin creates a receptionist.
2. Node admin publishes a package; it appears in the app.
3. Customer books and pays online; receptionist confirms the arrival.
4. Online and walk-in race for the last room: exactly one wins.
5. Island disconnected: packages unavailable online, walk-ins continue; reconnect and show sync.
6. Show benchmark charts: search speedup and report speedup.

17. Risks and mitigation

Risk	Impact	Mitigation
Hotspot blocks device-to-device traffic or device limit reached	High	Test in week 1; spare pocket router; fewer phones if needed
No internet at the demo venue (Supabase unreachable)	High	Islands still work; prepare mobile data; local fallback for central storage
Expo Go version mismatch on a phone (especially iPhone)	Medium	Pin SDK 58; check every phone in week 1
Laptop IP changes on reconnect	Medium	Re-check `ipconfig` before demo; update network config
Scope too large for the deadline	High	Prioritise: booking, arrival, offline demo, experiments; cut reviews/maps/extras
Integration problems between team members' parts	Medium	Agree on the gRPC contract first; integrate weekly on `main`
Hold expiry or clock differences between laptops	Low	Expiry decided only by the island's own clock
Secret key leaked	Medium	Key only in central `.env`; rotate in dashboard; legacy keys disabled

18. Project plan and team

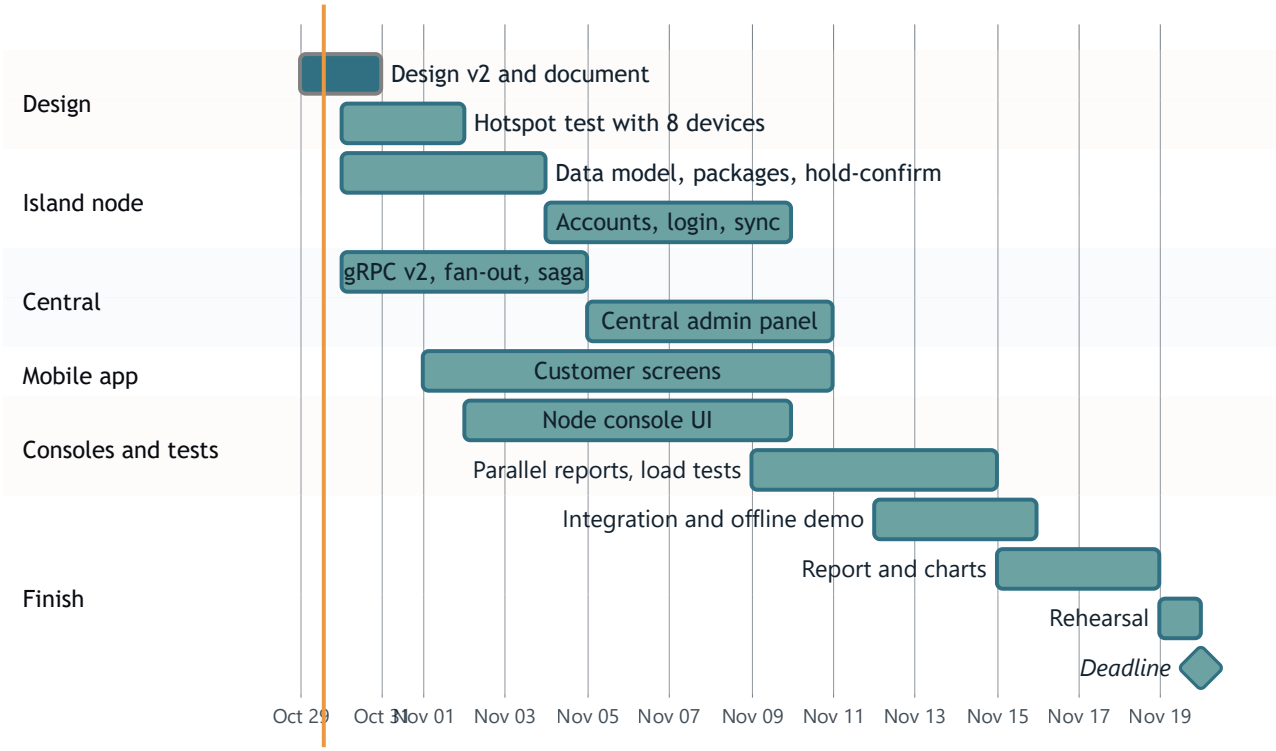


Figure 14. Schedule from 29 October to the 20 November deadline

Member	Responsibilities
Member A — Laptop 1	Central server, booking saga, central admin panel, Supabase, gRPC contract, integration
Member B	Island node: services, packages, inventory, hold/confirm, accounts and login, outbox and directory sync
Member C	Mobile app: all customer screens
Member D	Node console UI (receptionist and node admin), parallel reports, load tests, benchmark charts

Current status (29 October 2026)

- Working prototype of v1: island nodes, central server, gRPC, Supabase, parallel search (2.93× speedup), race and partition tests passing.
- Design revised to v2 after adviser feedback (this document); code rework starts with the island data model and packages.

19. Open questions for the adviser

#	Question	Proposed answer
1	Is a simulated payment acceptable, or is a sandbox gateway (e.g. GCash/PayMongo test mode) expected?	Simulated payment with success/failure option
2	Package price per package (up to N guests) or per person?	Per package, with a maximum number of guests
3	Cancellation and refund rules?	Full refund up to 48 h before the date
4	How long should an unpaid online hold last?	10 minutes
5	Should the central admin approve node admins' package and price changes before they go live?	No — node admins publish directly; central sees all changes
6	Which reports are required (occupancy, sales, cancellations, no-shows)?	Occupancy and sales per island and package
7	Is "unavailable online while the island is offline" acceptable as the partition behaviour?	Yes — consistency over availability (§12.3)
8	Do customers need accounts?	No — reference + e-mail look-up